The `Integer` class in Java is a wrapper class for the primitive data type `int`. It is part of the `java.lang` package and provides various methods to work with integers, such as parsing, converting, and comparing them. The `Integer` class also provides constants and static methods for integer operations.

Key Features of Integer Wrapper Class

1. **Immutability**: Instances of the `Integer` class are immutable.
2. **Constants**: Provides constants like `MIN_VALUE` and `MAX_VALUE`.
3. **Static Methods**: Methods for parsing strings to integers, comparing integers, and converting integers to strings.

Integer Class Methods

Here are some commonly used methods in the `Integer` class:

1. `int intValue()`
2. `static int compare(int x, int y)`
3. `boolean equals(Object obj)`

4. `static int parseInt(String s)`
5. `static int parseInt(String s, int radix)`
6. `String toString()`
7. `static String toString(int i)`
8. `static Integer valueOf(int i)`
9. `static Integer valueOf(String s)`
10. `static Integer valueOf(String s, int radix)`
11. `static int sum(int a, int b)`

1. `int intValue()`

Returns the value of this `Integer` as an `int`.

```
public class IntegerExample {
    public static void main(String[] args) {
        Integer i = Integer.valueOf(10);
        int value = i.intValue();
        System.out.println(value);  // Output: 10
    }
}
```

Output:

```
10
```

2. `static int compare(int x, int y)`

Compares two `int` values numerically.

```
public class IntegerExample {
    public static void main(String[] args) {
        System.out.println(Integer.compare(10, 20));  // Output: -1
    }
}
```

Output:

```
-1
```

3. `boolean equals(Object obj)`

Compares this object to the specified object.

```
public class IntegerExample {
```

```
    public static void main(String[]
args) {
        Integer i1 =
Integer.valueOf(10);
        Integer i2 =
Integer.valueOf(10);

System.out.println(i1.equals(i2));  //
Output: true
    }
}
```

Output:

```
true
```

4. `static int parseInt(String s)`

Parses the string argument as a signed decimal integer.

```
public class IntegerExample {
    public static void main(String[]
args) {
        int i =
Integer.parseInt("123");
```

```
        System.out.println(i);  // 
Output: 123
    }
}
```

Output:

```
123
```

5. `static int parseInt(String s, int radix)`

Parses the string argument as a signed integer in the radix specified by the second argument.

```
public class IntegerExample {
    public static void main(String[] 
args) {
        int i = Integer.parseInt("7B", 
16);
        System.out.println(i);  // 
Output: 123
    }
}
```

Output:

```
123
```

6. `String toString()`

Returns a string representation of this `Integer` object.

```
public class IntegerExample {
    public static void main(String[]
args) {
        Integer i =
Integer.valueOf(123);

System.out.println(i.toString());  //
Output: "123"
    }
}
```

Output:

```
123
```

7. `static String toString(int i)`

Returns a string representation of the integer argument.

```
public class IntegerExample {
    public static void main(String[] args) {
        String s = Integer.toString(123);
        System.out.println(s);   // Output: "123"
    }
}
```

Output:

```
123
```

8. `static Integer valueOf(int i)`

Returns an `Integer` instance representing the specified `int` value.

```
public class IntegerExample {
    public static void main(String[] args) {
        Integer i = Integer.valueOf(123);
        System.out.println(i);   // Output: 123
    }
```

```
}
```

Output:

```
123
```

9. `static Integer valueOf(String s)`

Returns an `Integer` object holding the value of the specified `String`.

```
public class IntegerExample {
    public static void main(String[]
args) {
        Integer i =
Integer.valueOf("123");
        System.out.println(i);  //
Output: 123
    }
}
```

Output:

```
123
```

10. `static Integer valueOf(String s, int radix)`

Returns an `Integer` object holding the value extracted from the specified `String` when parsed with the radix given by the second argument.

```
public class IntegerExample {
    public static void main(String[] args) {
        Integer i = Integer.valueOf("7B", 16);
        System.out.println(i);  // Output: 123
    }
}
```

Output:

```
123
```

11. `static int sum(int a, int b)`

Adds two integers together as per the + operator.

```
public class IntegerExample {
```

```
    public static void main(String[]
args) {
        int sum = Integer.sum(10, 20);
        System.out.println(sum);  //
Output: 30
    }
}
```

Output:

```
30
```